AdaBoost in Machine Learning

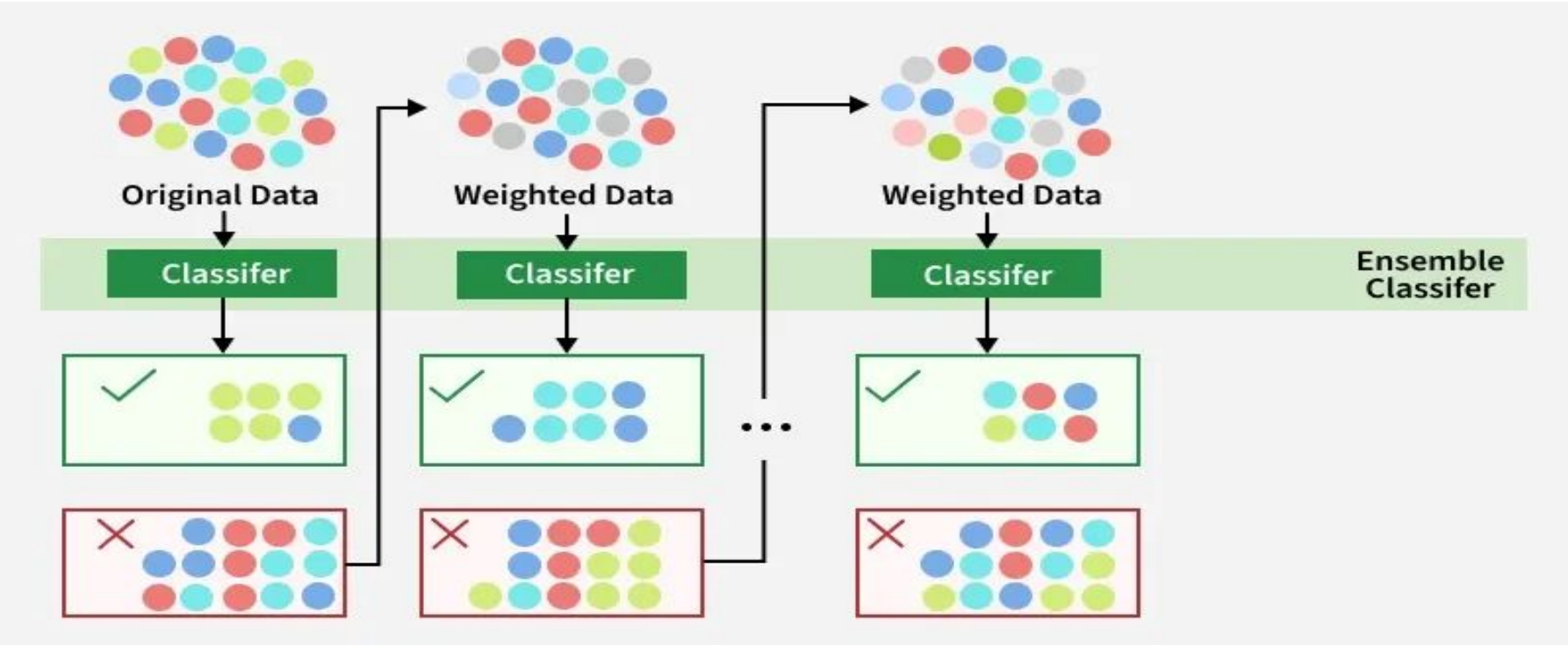
AdaBoost is a boosting technique that combines several weak classifiers in sequence to build a strong one.

Each new model focuses on correcting the mistakes of the previous one until all data is correctly classified or a set number of iterations is reached.

Think of it like in a class, a teacher focuses more on weak learners to improve its academic performance, similarly boosting work.

AdaBoost (Adaptive Boosting) assigns equal weights to all training samples initially and iteratively adjusts these weights by focusing more on misclassified data points for the next model.

It effectively reduces bias and variance making it useful for classification tasks but it can be sensitive to noisy data and outliers.



Step 1: Initial Model (B1)

- The dataset consists of multiple data points (red, blue and green circles).
- Equal weight is assigned to each data point.
- The first weak classifier attempts to create a decision boundary.
- 8 data points are wrongly classified.

Step 2: Adjusting Weights (B2)

- The misclassified points from B1 are assigned higher weights (shown as darker points in the next step).
- A new classifier is trained with a refined decision boundary focusing more on the previously misclassified points.
- Some previously misclassified points are now correctly classified.
- 6 data points are wrongly classified.

Step 3: Further Adjustment (B3)

- The newly misclassified points from B2 receive higher weights to ensure better classification.
- The classifier adjusts again using an improved decision boundary and 4 data points remain misclassified.

Step 4: Final Strong Model (B4 - Ensemble Model)

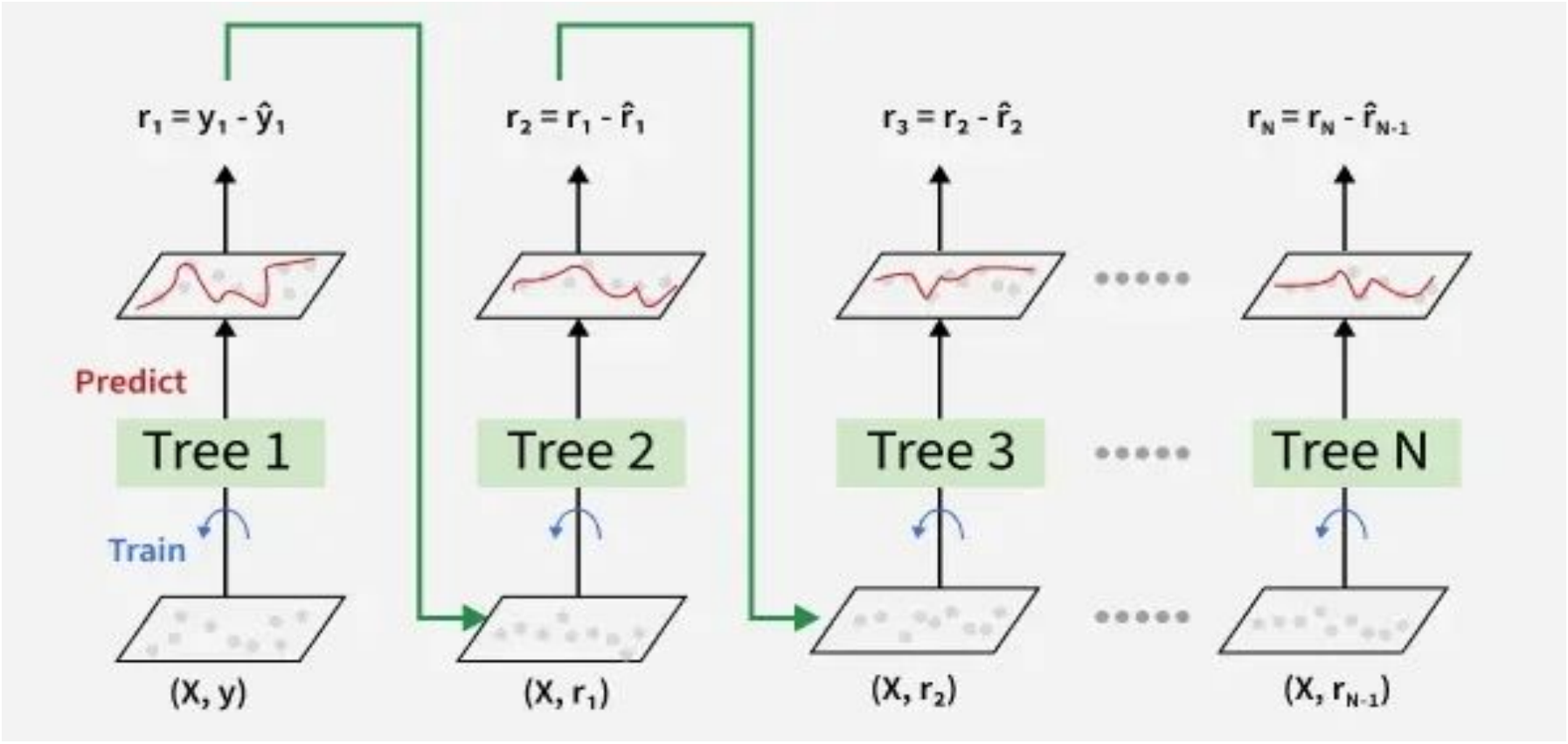
- The final ensemble classifier combines B1, B2 and B3 to get strengths of all weak classifiers.
- By aggregating multiple models the ensemble model achieves higher accuracy than any individual weak model.

Gradient Boosting in ML

Gradient Boosting is a **boosting** algorithm and here each new model is trained to minimize the loss function such as mean squared error or cross-entropy of the previous model using gradient descent.

In each iteration the algorithm computes the gradient of the loss function with respect to predictions and then trains a new weak model to predict this gradient.

Predictions of the new model are then added to the ensemble (all models prediction) and the process is repeated until a stopping criterion is met


$r_1 = y_1 - \hat{y}_1$
$r_2 = r_1 - \hat{r}_1$
$r_3 = r_2 - \hat{r}_2$
$r_N = r_N - \hat{r}_{N-1}$
Predict
Tree 1
Tree 2
Tree 3
Tree N
Train
(X, y)
(X, r_1)
(X, r_2)
(X, r_{N-1})

3. **Shrinkage**

After each tree is trained its predictions are **shrunk** by multiplying them with the learning rate η which ranges from 0 to 1.

This prevents overfitting by ensuring each tree has a smaller impact on the final model.

Once all trees are trained predictions are made by summing the contributions of all the trees. The final prediction is given by the formula:

$$y_{pred} = y_1 + \eta \cdot r_1 + \eta \cdot r_2 + \cdots + \eta \cdot r_N$$

Shrinkage and Model Complexity

A key feature of Gradient Boosting is shrinkage which scales the contribution of each new model using **learning rate**

- **Smaller learning rates:** mean the contribution of each tree is smaller which reduces the risk of overfitting but requires more trees to achieve the same performance.
- **Larger learning rates:** mean each tree has a more significant impact but this can lead to overfitting.

There's a trade off between the learning rate and the number of estimators (trees) a smaller learning rate usually means more trees are required to achieve optimal performance.

Implementing Gradient Boosting for Classification and Regression

- **n_estimators:** This specifies the number of trees (estimators) to be built. A higher value typically improves model performance but increases computation time.
- **learning_rate:** This is the shrinkage parameter. It scales the contribution of each tree.
- **random_state:** It ensures reproducibility of results. Setting a fixed value for random_state ensure that you get the same results every time you run the model.
- **max_features**: This parameter limits the number of features each tree can use for splitting. It helps prevent overfitting by limiting the complexity of each tree and promoting diversity in the model.

Features	AdaBoost	Gradient Boosting
Weight Update Strategy	Increase weights of misclassified sample so that the next learner focuses more on them.	Updates predictions by minimizing a loss function using the negative gradient
Base learners	AdaBoost uses simple decision trees with one split known as the decision stumps of weak learners.	Gradient Boosting can use a wide range of base learners such as decision trees and linear models.
Sensitivity to Noise	AdaBoost is more sensitive to noisy data and outliers due to aggressive weighting.	Gradient Boosting is less sensitive as it smooths updates using gradients.

XGBoost

Traditional machine learning models like decision trees and random forests are easy to interpret but often struggle with accuracy on complex datasets.

XGBoost short form for eXtreme Gradient Boosting is an advanced machine learning algorithm designed for efficiency, speed and high performance.

- XGBoost uses **decision trees** as its base learners and combines them sequentially to improve the model's performance. Each new tree is trained to correct the errors made by the previous tree and this process is called boosting.

- It has built-in parallel processing to train models on large datasets quickly. XGBoost also supports customizations allowing users to adjust model parameters to optimize performance based on the specific problem.

What Makes XGBoost "eXtreme"?

XGBoost incorporates several techniques to reduce overfitting and improve model generalization:

- **Learning rate (eta):** Controls each tree’s contribution; a lower value makes the model more conservative.
- **Regularization:** Adds penalties to complexity to prevent overly complex trees.
- **Pruning:** Trees grow depth-wise, and splits that do not improve the objective function are removed, keeping trees simpler and faster.
- **Combination effect:** Using learning rate, regularization, and pruning together enhances robustness and reduces overfitting.

Handling Missing Data

XGBoost manages missing values robustly during training and prediction using a sparsity-aware approach.

- **Sparsity-Aware Split Finding:** Treats missing values as a separate category when evaluating splits.
- **Default direction:** During tree building, missing values follow a default branch.
- **Prediction:** Instances with missing features follow the learned default branch.
- **Benefit:** Ensures robust predictions even with incomplete input data.

Advantages of XGBoost

XGBoost includes several features and characteristics that make it useful in many scenarios:

- Scalable for large datasets with millions of records.
- Supports parallel processing and GPU acceleration.
- Offers customizable parameters and regularization for fine-tuning.
- Includes feature importance analysis for better insights.
- Available across multiple programming languages and widely used by data scientists

Disadvantages of XGBoost

XGBoost also has certain aspects that require caution or consideration:

- Computationally intensive; may not be suitable for resource-limited systems.
- Sensitive to noise and outliers; careful preprocessing required.
- Can overfit, especially on small datasets or with too many trees.
- Limited interpretability compared to simpler models, which can be a concern in fields like healthcare or finance.